



Under the hood: state, gates, and the request lifecycle

How a single run moves through consent, agents, guardrails, and persistence

A LangGraph StateGraph with a real parallel fan-out/fan-in, two conditional human-in-the-loop entry points, a deterministic authoritative verifier, and every store the pipeline touches — from the in-memory checkpointer to the two durable stores (Chroma, SQLite).

StateGraph

Fan-out/fan-in

Two HITL entry points

Typed fault taxonomy

Cost-tracked calls

REQUEST LIFECYCLE

- consent_gate** reads Tier-1 consent; assigns `run_id` from the graph's `thread_id`, threaded into every downstream log event.
- rate_limit_gate** checks per-user image/day and analysis/day counters (in-memory, reset on restart).
- vision_agent** → **nutrition_agent**: LLM maps food text/photo to canonical keys; a deterministic lookup table sums calories/macros — never an LLM guess.
- Parallel fan-out**: `medical_rag_agent`, `activity_agent`, `sms_agent`, `calendar_agent`, and `lab_report_agent` all run concurrently off `nutrition_agent`'s output.
- trend_agent** (no LLM) reads the last 90 days of glucose/weight/sleep from SQLite and classifies each trend before any model call happens.
- prediction_agent** emits SAFE-category-only predictions; each is checked against an allowlist and confidence-capped before `guardrail_gate` decides HITL vs. `recommendation_agent`.
- recommendation_agent** → **guardrail_verifier**: the deterministic verifier re-checks categories, confidence, allergens, calorie/exercise bounds, and citation ownership directly from state — independent of what any LLM concluded.
- critic_agent** (advisory) only runs on a verifier-clean path; **finalize** composes the report, appends the mandatory disclaimer, and sets `run_status` to success or degraded.

TWO HUMAN-IN-THE-LOOP ENTRY POINTS



A `reject` decision from either entry point routes straight to `finalize` with no output shown to the user.

DATA STORES & PERSISTENCE



Chroma vector store

Medical-document embeddings, chunked by heading + token budget. Filtered by `user_id` metadata at query time; deletable per-user for consent revocation. Durable across restarts.



SQLite metrics store

Timestamped glucose/weight/sleep readings, one table, indexed by (`user_id`, `metric_name`, `recorded_at`). The only store that historic trend reasoning can rely on. Durable across restarts.



JSONL audit log + docs report

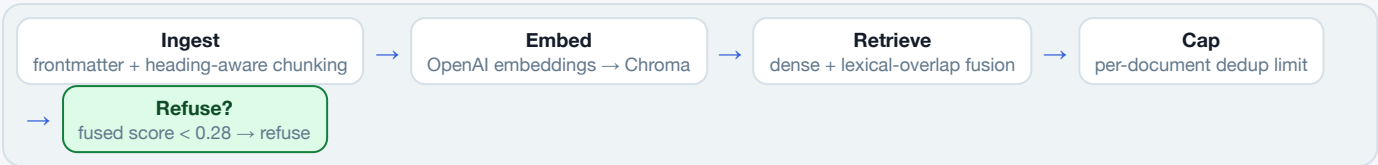
Every agent emits a structured event (`run_id`, `model`, `tokens`, `cost`, `stop_reason`, `fault`) appended to `logs/audit.jsonl`; `reporting.py` renders it to `docs/audit_trace_report.md` and console — never a live UI table.



InMemoryStore + MemorySaver

LangGraph long-term Store (preferences, progress notes) and the per-thread checkpointer that makes `interrupt()/resume` possible. Both reset on process restart — explicitly out of scope for this build stage.

■ **RAG RETRIEVAL FLOW**



Failure attribution

- **Typed fault, not a string.** Every exception maps to a `Fault(layer, fault_class, message, model, stop_reason, request_id)` instead of a free-text error.
- `classify_exception()` infers timeout / content_filter / provider_error / schema_validation from the raw exception when a call site doesn't already know.
- **Recovered faults are counted, not hidden.** finalize sets `run_status="degraded"` and a `recovered_faults` count whenever any agent fell back gracefully.

Cost & replay instrumentation

- `call_structured()` is the single choke point for every model call: validates stop_reason, computes cost from a pinned per-model price table, and enforces the per-run spend cap before calling.
- **Every event carries** run_id, resolved model id, prompt version, tokens, cost_usd, stop_reason, and provider_request_id — enough to replay a failure locally.
- **Prompts are versioned artifacts** (prompts.py), not inline string literals, so a completion can be attributed to the exact prompt that produced it.

■ **AT A GLANCE**

2 durable stores: Chroma + SQLite	2 HITL re-entry points, tracked by review_stage	0.28 fused-score refusal threshold for RAG	6 typed fault classes across the pipeline	1 authoritative deterministic verifier
---	---	--	---	--